

```
>> A = 5 1 2; -3 0 1; 2 4 1;
```

```
A = 5 1 2; -3 0 1; 2 4 1;
```

↑

Error: Invalid expression. Check for missing multiplication operator, missing or unbalanced delimiters, or other syntax error. To construct matrices, use brackets instead of parentheses.

```
>> A = [5 1 2; -3 0 1; 2 4 1];
```

```
>> B = [4*2 2/3; 1/201 5-8.2; 0.00001 (9+4)/3;]
```

```
B =
```

```
8.0000    0.6667
0.0050   -3.2000
0.0000    4.3333
```

```
>> A(2,3), B(3,2), B(1,2)
```

```
ans =
```

```
1
```

```
ans =
```

```
4.3333
```

```
ans =
```

```
0.6667
```

```
>> A(1,:), A(:,3), B(2,:)
```

```
ans =
```

```
5    1    2
```

```
ans =
```

```
2
1
1
```

```
ans =
```

```
0.0050   -3.2000
```

```
>> format long
>> B
```

```
B =

    8.000000000000000    0.666666666666667
    0.004975124378109   -3.199999999999999
    0.000010000000000    4.333333333333333
```

```
>> format short
>> B
```

```
B =

    8.0000    0.6667
    0.0050   -3.2000
    0.0000    4.3333
```

```
>> A = [1 1/2; 1/3 1/4; 1/5 1/6], B = [5 -2], C = [4 5/4 9/4; 1 2 3]
```

```
A =

    1.0000    0.5000
    0.3333    0.2500
    0.2000    0.1667
```

```
B =

     5     -2
```

```
C =

    4.0000    1.2500    2.2500
    1.0000    2.0000    3.0000
```

```
>> A * C
```

```
ans =

    4.5000    2.2500    3.7500
    1.5833    0.9167    1.5000
    0.9667    0.5833    0.9500
```

```
>> A*B
```

```
Error using *
Incorrect dimensions for matrix multiplication. Check
that the number of columns in the first matrix
matches the number of rows in the second matrix. To
perform elementwise multiplication, use '.*'.
```

```
>> A * B
```

```
Error using *  
Incorrect dimensions for matrix multiplication. Check  
that the number of columns in the first matrix  
matches the number of rows in the second matrix. To  
perform elementwise multiplication, use '.*'.
```

```
>> A + C'
```

```
ans =
```

```
5.0000    1.5000  
1.5833    2.2500  
2.4500    3.1667
```

```
>> B * A - C' * A
```

```
Error using *  
Incorrect dimensions for matrix multiplication. Check  
that the number of columns in the first matrix  
matches the number of rows in the second matrix. To  
perform elementwise multiplication, use '.*'.
```

```
>> (2 * C - 6 * A') * B'
```

```
Error using *  
Incorrect dimensions for matrix multiplication. Check  
that the number of columns in the first matrix  
matches the number of rows in the second matrix. To  
perform elementwise multiplication, use '.*'.
```

```
>> A * C - C * A
```

```
Matrix dimensions must agree.
```

```
>> A * A' + C' * C
```

```
ans =
```

```
18.2500    7.4583   12.2833  
7.4583    5.7361    8.9208  
12.2833    8.9208   14.1303
```

```
>> 2x + 4y + 6z = -12
```

```
2x + 4y + 6z = -12
```

```
↑
```

```
Error: Invalid expression. Check for missing  
multiplication operator, missing or unbalanced  
delimiters, or other syntax error. To construct  
matrices, use brackets instead of parentheses.
```

```
>> [2x + 4y + 6z = -12; 2x - 3y - 4z = 15; 3x + 4y + 5z = -8]
```

```
[2x + 4y + 6z = -12; 2x - 3y - 4z = 15; 3x + 4y + 5z = -8]
```

↑

Error: Invalid expression. Check for missing multiplication operator, missing or unbalanced delimiters, or other syntax error. To construct matrices, use brackets instead of parentheses.

```
>> [2*x + 4*y + 6*z = -12; 2*x - 3*y - 4*z = 15; 3*x + 4*y + 5*z = -8]
    [2*x + 4*y + 6*z = -12; 2*x - 3*y - 4*z = 15; 3*x + 4*y + 5*z = -8]
```

↑

Error: Incorrect use of '=' operator. To assign a value to a variable, use '='. To compare values for equality, use '=='.
Error in assignment operator: '='.

```
>> [2*x + 4*y + 6*z == -12; 2*x - 3*y - 4*z == 15; 3*x + 4*y + 5*z == -8]
Unrecognized function or variable 'x'.
```

```
>> A = [2 4 6;2 -3 4;3 4 5]
```

A =

2	4	6
2	-3	4
3	4	5

```
>> b = [-12 15 -8]'
```

b =

-12
15
-8

```
>> aug = [A b]
```

aug =

2	4	6	-12
2	-3	4	15
3	4	5	-8

```
>> diag
```

Error using diag
Not enough input arguments.

```
>> diag [1 2 3 4]
```

ans =

9×9 char array

'[]'

```

' 1      '
'      '
'      2  '
'      '
'      3  '
'      '
'      4  '
'      ] '

```

```
>> diag [ 0 1 1/2 1/3 1/4 ]
```

ans =

```
19x19 char array
```

```
[
  0
  1
  1
  /
  2
  1
  /
  3
  1
  /
  4
]
```

```
>> diag [ 5 5 5 5 5 ]
```

```
ans =
```

13×13 char array

' ['
 ' '
 ' 5
 ' '
 ' 5
 ' '
 ' 5
 ' '
 ' 5
 ' '
 ' '

```
'      5  '  
'      '  
'      ]'
```

```
>> diag [55555]
```

```
ans =
```

```
7×7 char array
```

```
'[      '  
' 5      '  
' 5      '  
' 5      '  
' 5      '  
' 5      '  
' 5      '  
'      ]'
```

```
>> A = [4 -2 1; 0 5 8]
```

```
A =
```

```
4      -2      1  
0       5      8
```

```
>> B = [-2 9 4; 7 -3 8]
```

```
B =
```

```
-2      9      4  
7      -3     8
```

```
>> A.*B
```

```
ans =
```

```
-8     -18      4  
0     -15     64
```

```
>> A./B and B./A
```

```
A./B and B./A
```

↑

Error: Invalid expression. Check for missing multiplication operator, missing or unbalanced delimiters, or other syntax error. To construct matrices, use brackets instead of parentheses.

```
>> A./B
```

```
ans =
```

-2.0000	-0.2222	0.2500
0	-1.6667	1.0000

>> B./A

ans =

-0.5000	-4.5000	4.0000
Inf	-0.6000	1.0000

>> A.^2

ans =

16	4	1
0	25	64

>> A = [1/2 1/3 1/4 1/5; 1/3 1/4 1/5 1/6; 1 1/2 1/3 1/4]

A =

0.5000	0.3333	0.2500	0.2000
0.3333	0.2500	0.2000	0.1667
1.0000	0.5000	0.3333	0.2500

>> reduce(A)

Check for missing argument or incorrect argument data
type in call to function 'reduce'.

>> reduce A

Check for missing argument or incorrect argument data
type in call to function 'reduce'.

>> reduce([A])

Check for missing argument or incorrect argument data
type in call to function 'reduce'.

>> reduce A

Check for missing argument or incorrect argument data
type in call to function 'reduce'.

>> A = [1/2 1/3 1/4 1/5; 1/3 1/4 1/5 1/6; 1 1/2 1/3 1/4]

A =

0.5000	0.3333	0.2500	0.2000
0.3333	0.2500	0.2000	0.1667
1.0000	0.5000	0.3333	0.2500

>> reduce(A)

Check for missing argument or incorrect argument data

type in call to function 'reduce'.

```
>> help reduce
```

```
--- help for lti/reduce ---
```

reduce Main interface to model approximation algorithms.

This function groups all the Hankel SV based model reduction routines (for continuous/discrete, stable/unstable models).

[GRED,REDINFO,GR] = reduce(G,ORDER,KEY1,VALUE1,KEY2,VALUE2,...) returns a reduced order model GRED of LTI object G and a predetermined error bound between the two models. An error bound is an indication of how "close" GRED is to G and is computed based on

additive error $\inf ||G1-G2||$, or
multiplicative error $\inf ||(G-GRED)/GRED||$ or
gap metrics (Ref.: ncfmr).

Hankel singular values of a stable system indicate the "states energy" of the system. Hence, reduced order can be directly determined by examining the system Hankel sv's. Model reduction routines, which based on Hankel sv's are grouped by their error bound types. In general, "additive" error methods are sufficient to provide a good reduced order model. But for systems with low damped poles and/or zeros, "multiplicative" error method (BSTMR) will minimize the "relative" error between G and GRED and produce a better fit on the low damped poles/zeros (see HANKELSV).

Examples:

```
[GRED,info]=reduce(G,5); % default to BALANCMR
[GRED,info]=reduce(G,[5:10],'ErrorType','mult'); % select BSTMR
[GRED,info]=reduce(G,{5:10},'algorithm','schur');%select SCHURMR
[GRED,info]=reduce(G,'ErrorType','mult','MaxError',[0.8,0.5]);
[GRED,info]=reduce(G,'algorithm','hankel','maxerror',[0.8,0.6,0.3]);
[GRED,info]=reduce(G,'weight',{wt1,wt2},'order',[4:2:10]); % weighted
[GL,info,GR]=reduce(G,'ErrroType','ncf','order',[10:2:18]);
```

Inputs:

G - LTI system to be reduced (no other inputs will list/plot Hankel sv's and prompt for reduced order)

Optional inputs:

ORDER - an integer array with desired order of reduced model.
A batch run of [m:n] can be specified for a pack of reduced order models to be generated. The unstable part is kept as default.

KEY	VALUE	MEANING

'Algorithm'	'balance'	Default for 'add' (balancmr.m)
	'schur'	Option for 'add' (schurmr.m)

	'hankel'	Option for 'add' (hankelmr.m)
	'bst'	Default for 'mul' (bstmr.m)
	'ncf'	Default for 'ncf' (ncfmr.m)

'ErrorType'	'add'	Additive error (default)
	'mult'	minimize multiplicative error at G output
	'ncf'	NCF nugap error

'MaxError'	real no. or	Reduce to achieve H-infinity error
	vector	If present, 'MaxError' overrides ORDER input.

'Weights'	{Wout,Win} cell	Optional 1x2 cell array of LTI weights Wout
		(output)and Win(input); default is both
		identity; used only with 'ErrorType', 'add'.
		Weights have to be invertible.

'Display'	'off' or 'on'	Display HANKELSV plots (default 'off');
		'Display' overrides ORDER input, if present.

'Order'	integer/vector	Order of reduced model
	or cell array	use it if not specifying as 2nd argument

A serial of reduced order models can be generated by specifying $m = x:y$. By default, all the anti-stable part of a physical system is kept. Weights are used to emphasize or deemphasize some frequency range of interests for better reduced order model approximation of the original model.

'MaxError' can be specified in the same fashion after an 'ErrorType' is selected. In this case, reduced order will be determined when the sum of the tails of the Hankel sv's reaches the 'Max Error'.

Outputs:

```

GRED      - LTI reduced order system,
            or the left coprime for normalized coprime case
REDINFO   - a struct of REDINFO.ErrorBound, REDINFO.StabSV, and
            REDINFO.UnstabSV. For 'ncf' option, it becomes REDINFO.hsv, REDINFO.GL,
            REDINFO.GR

```

G, GRED, and GR are LTI objects, either continuous or discrete time.

See also balancmr, schurmr, bstmr, ncfmr, hankelmr, WTBALREAL.

>>